

✓ Sofia ML Regression 2026 Summer

```
import pandas as pd
```

```
train_file_path = '/content/drive/MyDrive/Sofia ML Regression 2026 Summer/sofia-ml-regression-2026-summer-private/sum
test_file_path = '/content/drive/MyDrive/Sofia ML Regression 2026 Summer/sofia-ml-regression-2026-summer-private/sum

try:
    df_train = pd.read_csv(train_file_path)
    print(f"Successfully loaded training data from {train_file_path}.")
    display(df_train.head())
except FileNotFoundError:
    print(f"Error: Training file at '{train_file_path}' was not found. Please check the path and try again.")
except Exception as e:
    print(f"An error occurred while loading the training dataset: {e}")

try:
    df_test = pd.read_csv(test_file_path)
    print(f"Successfully loaded test data from {test_file_path}.")
    display(df_test.head())
except FileNotFoundError:
    print(f"Error: Test file at '{test_file_path}' was not found. Please check the path and try again.")
except Exception as e:
    print(f"An error occurred while loading the test dataset: {e}")
```

Successfully loaded training data from /content/drive/MyDrive/Sofia ML Regression 2026 Summer/sofia-ml-regression-2026-summer-private/sum

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	...	x12	
0	6.068698	-0.330868	2.141601	-0.303288	8.382691	-1.911467	-16.510225	6.699953	7.526628	-0.500969	...	15.024857	8.681
1	27.462096	0.633254	8.818176	-2.925587	10.120454	1.936762	-8.206700	9.626216	2.189313	3.671586	...	0.529507	-15.306
2	6.347459	-4.329401	4.630901	0.293031	-10.921375	-3.540781	11.792638	29.882761	-0.214754	1.194087	...	-16.139854	0.746
3	3.208970	3.765250	2.011389	0.748147	-18.280035	NaN	26.920570	5.506490	4.435851	1.441199	...	-7.786529	5.923
4	25.896142	-1.500400	0.977432	-0.836908	9.531903	-0.464663	-1.745255	-2.161853	-7.012604	3.033782	...	12.997261	1.707

5 rows x 22 columns

Successfully loaded test data from /content/drive/MyDrive/Sofia ML Regression 2026 Summer/sofia-ml-regression-2026-summer-private/sum

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	...	x11	
0	-19.541338	0.251163	7.835783	0.282576	8.649919	7.815808	-20.794369	13.328533	-7.217923	1.260250	...	7.401634	-8.300
1	3.269554	3.006026	12.599081	-2.142957	6.972854	0.942758	-0.455802	0.185824	-2.942602	-2.275020	...	1.447264	-4.645
2	-20.709421	-5.231539	-3.169028	2.064075	-15.085853	4.266250	-6.852488	-24.121770	2.222776	5.268725	...	3.627777	-3.067
3	11.195211	-0.584093	10.635770	0.242581	-7.741640	7.765390	19.363690	2.417049	0.096356	6.059145	...	8.524414	6.315
4	6.074311	-1.872913	21.688982	-0.833080	1.669815	-2.958456	-11.417641	-0.644289	-3.563414	0.461862	...	0.681104	0.269

5 rows x 21 columns

```
from sklearn.preprocessing import StandardScaler
import numpy as np
```

```
def data_processing(df, scaler=None, is_train=True):
    df_processed = df.copy()

    # 1. Identify feature columns
    cols_to_scale = [col for col in df_processed.columns if col not in ['Id', 'target']]

    # 2. Impute NaNs using median
    df_processed[cols_to_scale] = df_processed[cols_to_scale].fillna(df_processed[cols_to_scale].median())

    # 3. Fit and transform on df_processed
    if is_train:
        scaler = StandardScaler()
        df_processed[cols_to_scale] = scaler.fit_transform(df_processed[cols_to_scale])
    return df_processed, scaler
```

```

else:
    if scaler is not None:
        df_processed[cols_to_scale] = scaler.transform(df_processed[cols_to_scale])
    return df_processed

def cap_outliers_iqr(df, factor=1.5):
    """
    Caps numerical features using the Interquartile Range (IQR) method.
    Values below Q1 - 1.5*IQR or above Q3 + 1.5*IQR are clipped to the boundaries.
    """
    df_capped = df.copy()

    # Identify feature columns (excluding Id and target)
    feature_cols = [col for col in df_capped.columns if col not in ['Id', 'target']]

    for col in feature_cols:
        Q1 = df_capped[col].quantile(0.25)
        Q3 = df_capped[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - (factor * IQR)
        upper_bound = Q3 + (factor * IQR)

        # Clip values outside the boundaries
        df_capped[col] = np.clip(df_capped[col], lower_bound, upper_bound)

    return df_capped

# Apply outlier capping on raw training data before scaling
df_train_no_outliers = cap_outliers_iqr(df_train)

# Re-run data_processing with capped features
df_train_processed, fitted_scaler = data_processing(df_train_no_outliers, is_train=True)

```

```
df_train.describe()
```

	x0	x1	x2	x3	x4	x5	x6	x7	x8	
count	2478.000000	2479.000000	2475.000000	2478.000000	2473.000000	2469.000000	2476.000000	2485.000000	2480.000000	2472.000000
mean	0.370076	0.073406	0.171521	-0.011232	-0.119634	-0.040149	0.100882	0.202261	-0.034692	-0.066111
std	12.180983	3.923268	10.458479	1.868386	9.757498	2.688212	14.163533	14.376643	4.289606	3.651111
min	-41.101992	-11.863109	-44.495944	-6.713979	-34.224046	-10.245958	-47.954815	-55.423112	-15.795069	-14.736211
25%	-7.708991	-2.608902	-7.006155	-1.300097	-6.919555	-1.851412	-9.780362	-9.637381	-2.860445	-2.458611
50%	0.629896	0.068784	0.375571	-0.021830	0.224000	-0.114095	-0.166829	0.267809	-0.058034	-0.073611
75%	8.377939	2.681841	7.259837	1.284668	6.622266	1.746053	9.465421	9.620466	2.912471	2.269011
max	45.473105	13.916111	33.730027	6.205738	27.217847	8.186229	59.323011	52.876684	15.874072	12.168711

8 rows x 22 columns

```

import matplotlib.pyplot as plt
import seaborn as sns

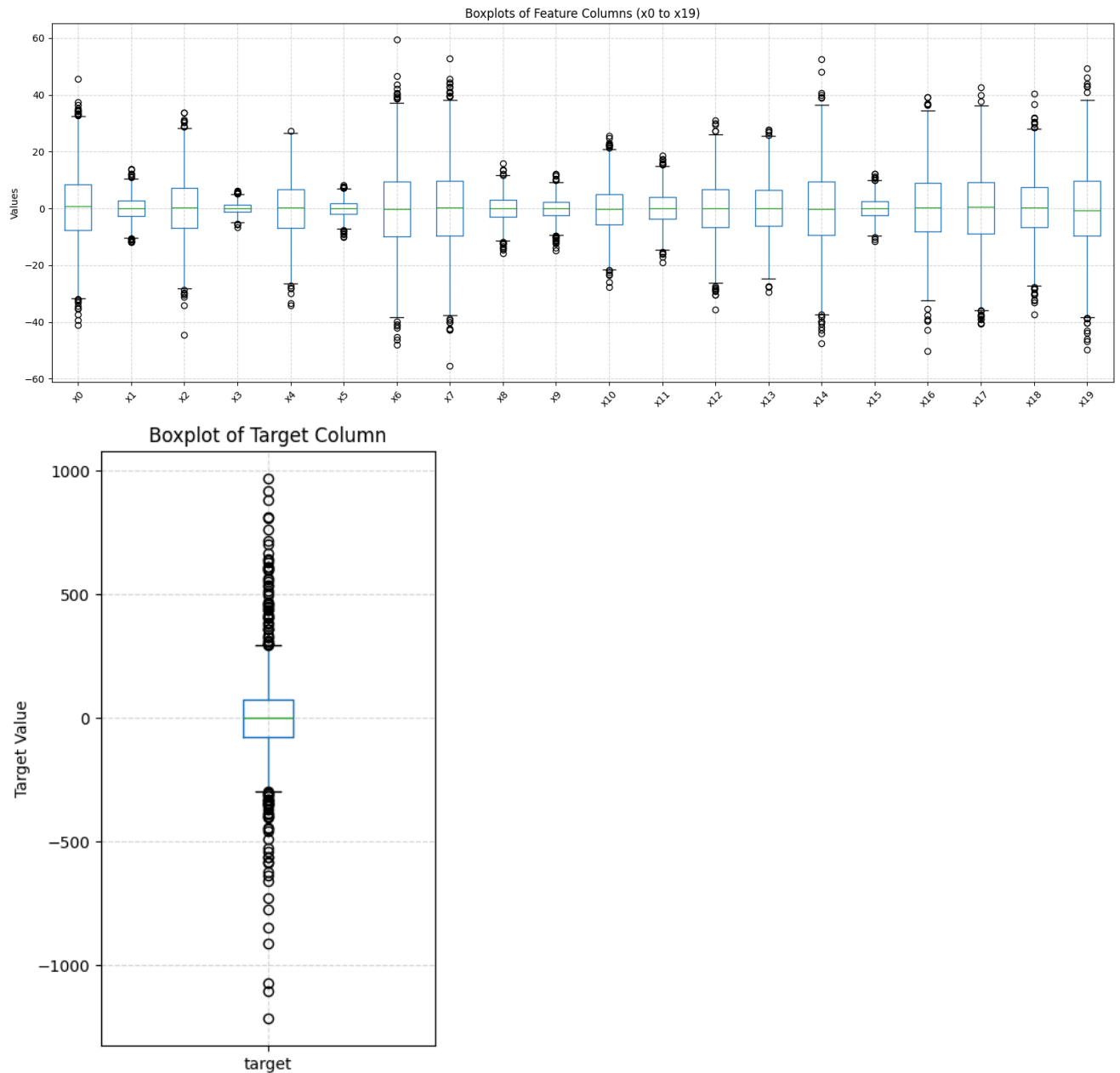
# 1. Identify feature and target columns (excluding 'Id')
cols_to_plot = [col for col in df_train.columns if col != 'Id']

# Option A: Plot feature columns on a single chart
plt.figure(figsize=(16, 6))
df_train[[c for c in cols_to_plot if c != 'target']].boxplot(rot=45)
plt.title('Boxplots of Feature Columns (x0 to x19)')
plt.ylabel('Values')
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

# Separate boxplot for 'target' due to higher scale range
plt.figure(figsize=(4, 6))
df_train[['target']].boxplot()
plt.title('Boxplot of Target Column')

```

```
plt.ylabel('Target Value')
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```



Summary: The target seems to have a lot of outliers as well. Maybe we need something to fit that too.

```
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Identify input feature columns (excluding 'Id' and 'target')
feature_cols = [col for col in df_train.columns if col not in ['Id', 'target']]

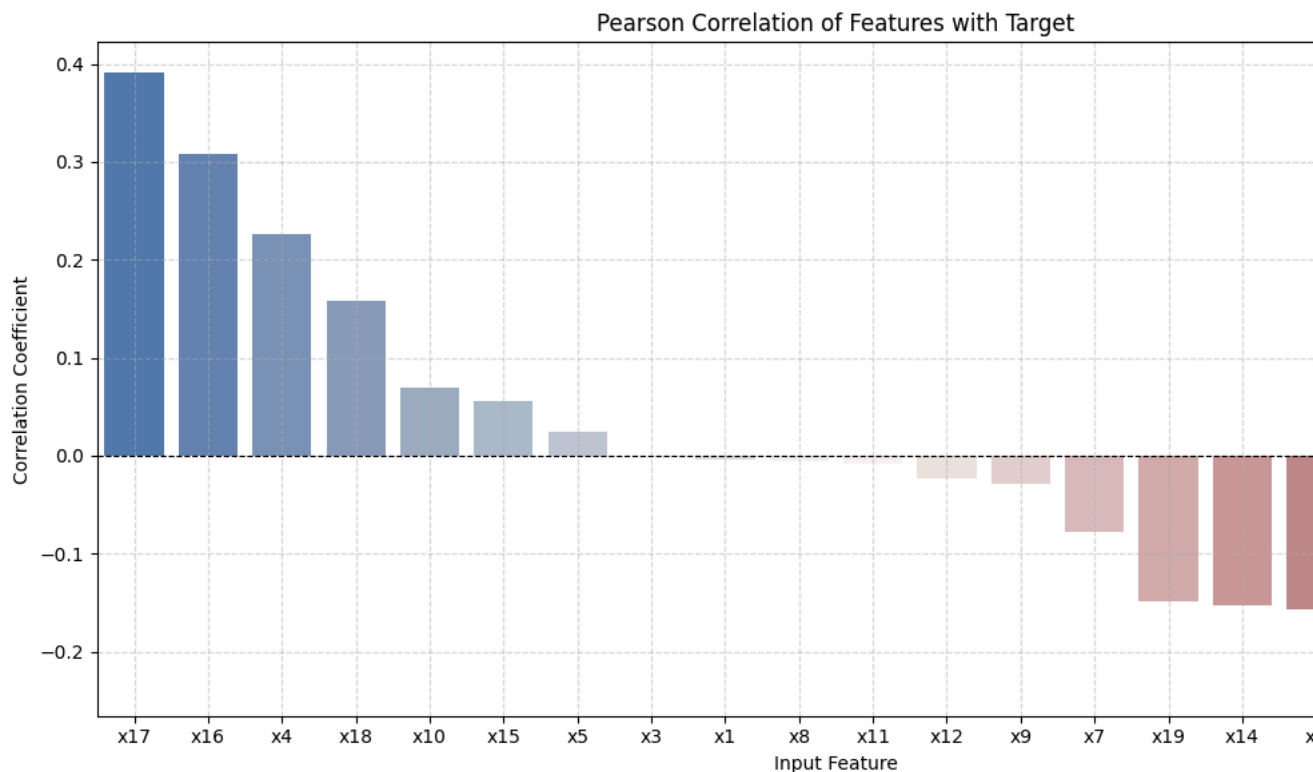
# 2. Compute Pearson correlation coefficients with 'target'
corr_with_target = df_train[feature_cols].apply(lambda x: x.corr(df_train['target'])).sort_values(ascending=False)

# 3. Plot ranked correlation values
plt.figure(figsize=(12, 6))
sns.barplot(x=corr_with_target.index, y=corr_with_target.values, palette='vlag')
```

```
plt.axhline(0, color='black', linewidth=0.8, linestyle='--')
plt.title('Pearson Correlation of Features with Target')
plt.xlabel('Input Feature')
plt.ylabel('Correlation Coefficient')
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_2789/3273867795.py:12: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue`
 sns.barplot(x=corr_with_target.index, y=corr_with_target.values, palette='vlag')



['x1', 'x5', 'x8', 'x9', 'x11', 'x12'] can be dropped

```
import pandas as pd, numpy as np
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score, RepeatedKFold

def load_data(train_path, test_path):
    df_train = pd.read_csv(train_path)
    df_test = pd.read_csv(test_path)
    print(f"Train shape: {df_train.shape}, Test shape: {df_test.shape}")
    return df_train, df_test

def drop_noise_features(df_train, df_test, features_to_drop):
    df_train_selected = df_train.drop(columns=features_to_drop)
    df_test_selected = df_test.drop(columns=features_to_drop)

    feature_columns = [col for col in df_train_selected.columns if col not in ['Id', 'target']]
    print(f"Features remaining: {feature_columns}")

    return df_train_selected, df_test_selected, feature_columns

def impute_features(df_train_selected, df_test_selected, feature_columns):
    imputer = SimpleImputer(strategy='median')

    X_train = imputer.fit_transform(df_train_selected[feature_columns])
```

```

X_test = imputer.transform(df_test_selected[feature_columns])

return X_train, X_test, imputer

def cross_validate_model(X_train, y_train, n_splits=5, n_repeats=20, random_state=42):
    model = LinearRegression()
    cv_splitter = RepeatedKFold(n_splits=n_splits, n_repeats=n_repeats, random_state=random_state)
    cv_scores = cross_val_score(model, X_train, y_train, cv=cv_splitter, scoring='r2')

    print(f"CV R2: {cv_scores.mean():.4f}")
    return cv_scores

def fit_final_model(X_train, y_train, feature_columns):
    model = LinearRegression()
    model.fit(X_train, y_train)

    coef_series = pd.Series(model.coef_, index=feature_columns).sort_values(key=abs, ascending=False)
    print(f"Intercept: {model.intercept_:.4f}")
    print("Coefficients (sorted by magnitude):")
    print(coef_series)

    return model

def generate_submission(model, X_test, df_test_selected):
    test_predictions = model.predict(X_test)

    df_submission = pd.DataFrame({
        'Id': df_test_selected['Id'],
        'target': test_predictions
    })
    print(f"Submission shape: {df_submission.shape}")

    return df_submission

def save_submission(df_submission, output_dir, filename='submission.csv'):
    submission_path = output_dir + filename
    df_submission.to_csv(submission_path, index=False)
    print(f"Saved to {submission_path}")
    return submission_path

# --- Cell 4: Load data ---
train_file_path = '/content/drive/MyDrive/Sofia ML Regression 2026 Summer/sofia-ml-regression-2026-summer-private/sum
test_file_path = '/content/drive/MyDrive/Sofia ML Regression 2026 Summer/sofia-ml-regression-2026-summer-private/sum

df_train, df_test = load_data(train_file_path, test_file_path)

features_to_drop = ['x1', 'x5', 'x8', 'x9', 'x11', 'x12']

df_train_selected, df_test_selected, feature_columns = drop_noise_features(
    df_train, df_test, features_to_drop
)

X_train, X_test, imputer = impute_features(df_train_selected, df_test_selected, feature_columns)
y_train = df_train_selected['target']

cv_scores = cross_validate_model(X_train, y_train)

model = fit_final_model(X_train, y_train, feature_columns)

df_submission = generate_submission(model, X_test, df_test_selected)
df_submission.head()

output_dir = '/content/drive/MyDrive/Sofia ML Regression 2026 Summer/sofia-ml-regression-2026-summer-private/'
save_submission(df_submission, output_dir)

```

```
Train shape: (2500, 22), Test shape: (2500, 21)
Features remaining: ['x0', 'x2', 'x3', 'x4', 'x6', 'x7', 'x10', 'x13', 'x14', 'x15', 'x16', 'x17', 'x18', 'x19']
CV R2: 0.5377
Intercept: 2.2109
Coefficients (sorted by magnitude):
x17    4.418105
x13   -3.528431
x4     3.456327
x16     3.436892
x2    -3.024521
x0    -2.906876
x3    -2.803426
x18     2.257948
x15     1.711451
x6    -1.680815
x14    -1.634576
x19    -1.538264
x10     0.985360
x7     -0.894048
dtype: float64
Submission shape: (2500, 2)
Saved to /content/drive/MyDrive/Sofia ML Regression 2026 Summer/sofia-ml-regression-2026-summer-private/submission.csv
'/content/drive/MyDrive/Sofia ML Regression 2026 Summer/sofia-ml-regression-2026-summer-private/submission.csv'
```